PROBLEM 2.4. **Project:** Application in speech recognition
For the project task please turn in a PDF report along with your code. It should contain all results (plots/ tables etc.) along with some discussion and interpretation thereof. Please also mention your hardships and how you overcame them.

a) Implement the EM Algorithm for an HMM. Take the transcriptions into account, by using the methods derived in the homework. Use `jax` heavily to accelerate execution. Use the formulation of the α-β algorithm that uses scaling factors, but beware of the mistake in [1, Eq. 13.65].

b) Verify that your implementation works using synthetic data generated by the provided Markov chain sampler `generate_chains.py`[1] and comparing the learned parameters with the ones used for sampling. Use a flat starting condition, i.e., make every state transition equally likely in the initial parameters.

Hint: For debugging purposes it may be interesting to look at the sums of γ and ξ and verify that they indeed constitute valid pmfs.

c) Apply the algorithm to the provided speech data. Calculate the features according to the presentation slides. Use the provided data loader for getting the features of the utterances along with their phonetic transcriptions. Learn the model using the first 700 sentences. Provide a plot of the evolution of LL over the iterations.

d) As a sanity check calculate the 5 most probable and 5 most improbable phoneme confusion pairs by means of the emission distributions. As a distance metric you may use the symmetrised Kullback-Leibler-divergence, mutual information or any other meaningful metric.

e) Check performance by means of the Viterbi algorithm on the remaining data. Fetch the 30 most probable transcriptions and their LLs for every test utterance. Calculate their Levenshtein distance to the ground truth. Average them – weighted with the softmax version of the corresponding LLs – and divide by the true sequence length to get a relative error measure. Provide a histogram of the relative errors across test data instances and compute their mean.

Keep in mind that we are applying a very rudimentary model to the data (Gaussians instead of mixtures as emission pdfs, exponential distributions for state transitions, no modelling whether transcription is corresponding to a possible sentence of real words etc.). Do not expect results better than 50% of errors. To convince yourself that the model is still significantly better than just guessing random sequences you might conduct the following experiment: Calculate the error rate that you get by just using random sequences generated by the provided Markov chain sampler using the estimated parameters. It should be significantly higher than the results from the Viterbi method.

[1] C. M. Bishop, *Pattern recognition and machine learning*. Springer, 2006.

[1]The third return value gives you the state sequence without duplicates, i.e. “transcriptions”.